

Seminario UNLP · 2026

Intro a IA Agéntica

De los fundamentos a *Sistemas Agénticos en Producción*

5 Módulos

LABS + EXPERIMENTOS

ESTADO DEL ARTE

BLOQUE 1 · MOTIVACIÓN

2026: un punto de inflexión

Proyecciones, tendencias, evidencia de ROI y la brecha del talento. Salteamos esta sección :)

BLOQUE 2 · EL ESTADO DEL ARTE

Un mes histórico

Julio de 2026

Un recorrido rápido por las arquitecturas,
patrones y herramientas que están definiendo el
estado del arte en 2026.

Mythos-Tier Model. Sonnet-Tier Habits.

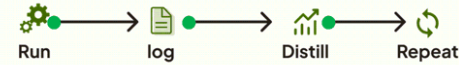
How to Actually Use Fable 5

Most people use Fable 5 like this



No memory. No learning. No compounding.

What actual Mythos do



Memory builds. System sharpens. Results compound.

The Four-Layer Architecture



The Golden Rule: Maker ≠ Verifier



The distinction that actually matters

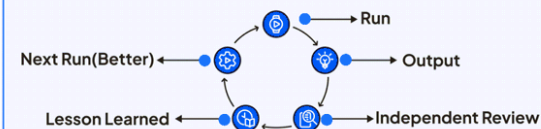
Self-learning

Self-learning = a model updating its own weights from experience. No public model, including Fable 5, does this today.

Self-improving

Self-improving = the system gets better over time. The model stays the same; the environment around it keeps improving.

The Self-Improvement Loop



Claude Fable 5 — Anthropic

Lanzado 9/6; suspendido 3 días por control de exportaciones tras un bypass de sus salvaguardas, restablecido globalmente el 1/7. Contexto de 1M tokens. "Adaptive thinking" reemplaza el razonamiento manual, con profundidad controlada por el parámetro `effort`.

GPT-5.6 Sol — OpenAI

Preview 26/6, lanzamiento 9/7. Modo Ultra: la decisión de descomponer una tarea en sub-agentes la toma el propio modelo, no el desarrollador. METR mide en Sol la tasa de *benchmark gaming* más alta de cualquier sistema testado. Over-Agency advertida por propios creadores. Contexto 1.5 M tokens

Gemini 3.5 Pro — Google

Preview, lanzamiento anunciado 17/7. Circulan claims de contexto de 2M y Deep Think auto-regulado. Tratar con cautela hasta documentación oficial de Google.

NOTA: el propio modelo decide cuándo descomponer una tarea en sub-agentes en paralelo (HTN + descomposición guiada por contexto), pero eso no elimina la orquestación — la mueve. Antes vivía del lado del desarrollador, escrita a mano con un framework como ADK o LandGraph.

AI APPLICATION PLANE

Deliver intelligence.

Everything that builds intelligent experiences and solves business problems.

- AI Agents
- RAG
- MCP
- LangGraph
- Prompt Engineering
- Tools & Workflows
- User Experiences

What Most People Talk About

AI CONTROL PLANE

Make intelligence operational.

The foundational layer that makes AI secure, governed, observable, cost-effective, and scalable at enterprise scale.

- Authentication**
Verify users, apps and workloads (Entra ID, IAM, Managed Identities)
- Authorization**
Control who can access which models, agents, tools and data
- Governance & Guardrails**
Enforce policies, safety, PII protection, content filtering and compliance
- Audit & Traceability**
Capture prompts, responses, actions, inputs, outputs and metadata
- Cost Management**
Budgets, quotas, rate limits, chargeback and showback
- Model Routing**
Select the right model for the right job across providers and regions
- Observability & Monitoring**
Performance, usage, quality, latency, errors and SLIs
- Knowledge & Data Platforms**
Connectors, catalogs, vector stores, indexing and retrieval governance
- SDKs & Developer Experience**
Simple, consistent, secure interfaces for all developers and languages

What Every Enterprise Eventually Needs

Users / Applications

Enterprise AI SDK

AI Gateway (Control Plane)

Model Providers

Enterprise Data & Knowledge

✓ Secure | ✓ Governed | ✓ Observable | ✓ Cost-Effective | ✓ Scalable

The 8 Models Powering 2026's Top AI Agents

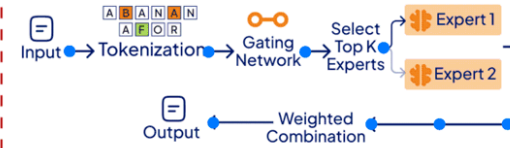
What every AI builder needs to know.

LLM (Large Language Model)



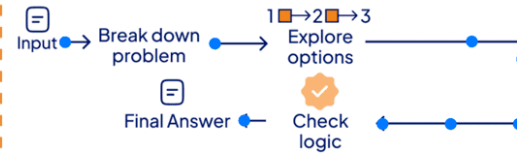
Example GPT-5.5 Claude 4 Opus Gemini 3.1 Pro

MoE (Mixture of Experts)



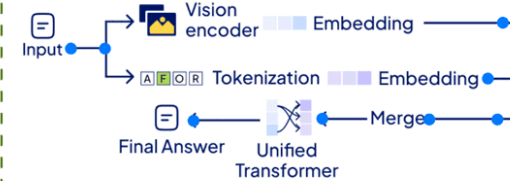
Example DeepSeek V3.2 Llama 4 Maverick

LRM (Large Reasoning Models)



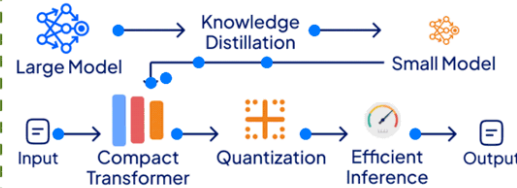
Example GPT-5.5 Thinking Claude Opus 4.6

VLM (Vision Language Model)



Example GPT-4o Advanced Qwen-VL-Max

SLM (Small Language Models)



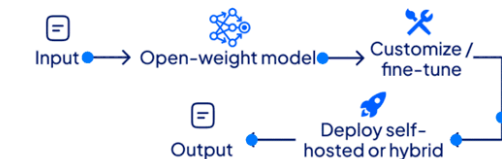
Example Gemma 3 (9B) Mistral Small 24B

Agentic - Action Model



Example OpenAI Agents API xLAM 2.0

Open-source Frontier Model



Example DeepSeek-R1 Qwen 3.572B

Specialized Model

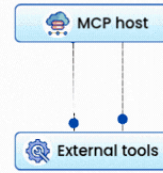


Example Cursor/Codex (coding) GitHub Copilot

12 AI CONCEPTS DEVELOPERS SHOULD KNOW

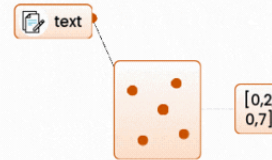


MCP



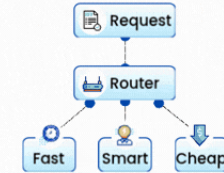
Standardizes how AI apps connect to data and tools

Embeddings



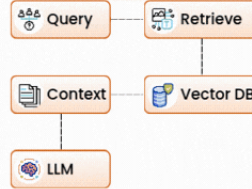
Turns text into vectors that capture meaning

Model routing



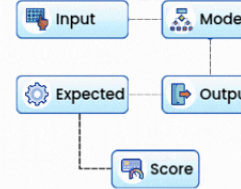
Picks the best model for each request (cost vs quality)

RAG



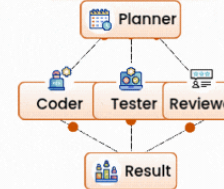
Fetches relevant info and adds it to the prompt

Evals



Compares output to expected results to track quality

Multi-agent



Specialized agents collaborate on complex problems

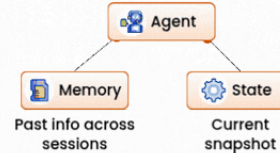
A2A protocol



Open standard for Agent-to-Agent communication across vendors

Agents discover and delegate to each other

Agent Memory



What the agent remembers vs what it's tracking now

Context Windows



The max tokens a model can process in a single request

Bigger = more it can "see" at once

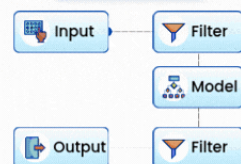
Fine-Tuning



Teach a base model your domain, tone, or task style

Use when prompts alone aren't enough

Guardrails



Rules and checks on what goes in and out

Blocks unsafe, off-topic, or leaky responses

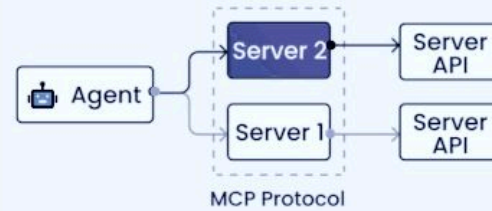
Tool Use



The model calls functions to act in the real world

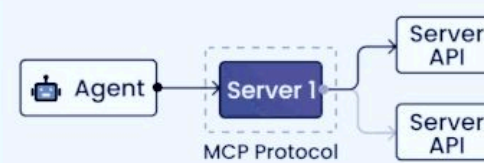
MCP Design Patterns

Best Design pattern based on your workflow



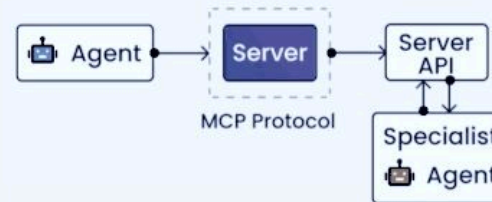
Direct API Wrapper Pattern

A 1:1 mapping between MCP tools and existing APIs. Useful for quick AI integration with stable APIs.



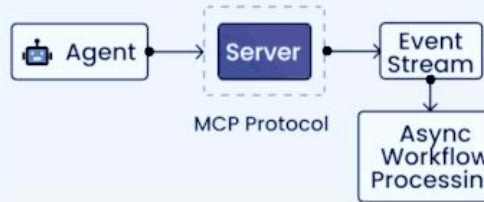
Composite Service Pattern

MCP tools orchestrate multiple APIs to provide higher-level capabilities. Ideal for complex workflows.



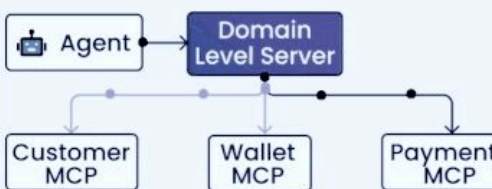
MCP-to-Agent Pattern

MCP tools invoke other AI agents for specialized reasoning. Enables modular AI architecture.



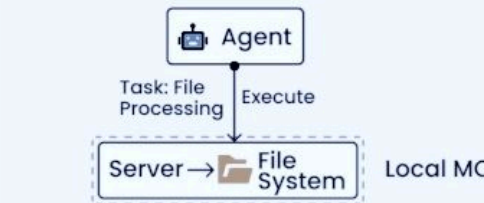
Event-Driven Integration Pattern

MCP tools publish events to trigger asynchronous processing. Great for non-blocking workflows.



Hierarchical MCP Pattern

MCP servers are organized in a hierarchy, with top-level routers managing cross-cutting concerns and sub-domain servers.



Local Resource Access Pattern

Secure access to local file systems for document processing and analysis. Offers high performance and offline capabilities.

Top 5 RAG Architectures

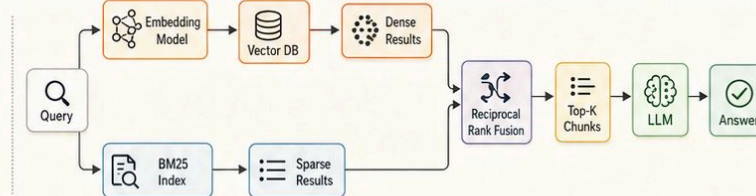
You Must Know in 2026

By Brij Kishore Pandey • @brijpandeyji

01

HYBRID RAG

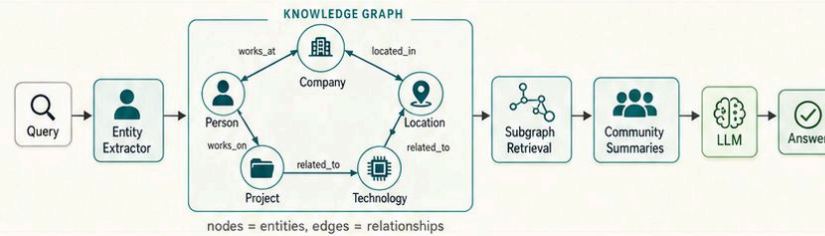
Dense vectors meet sparse keywords.



02

GRAPH RAG

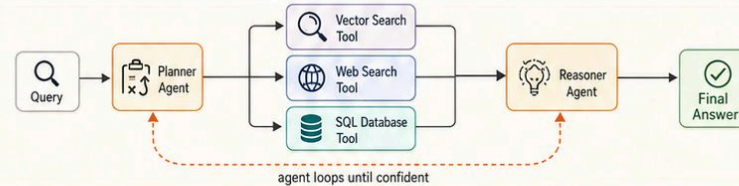
Answers live in the relationships.



03

AGENTIC RAG

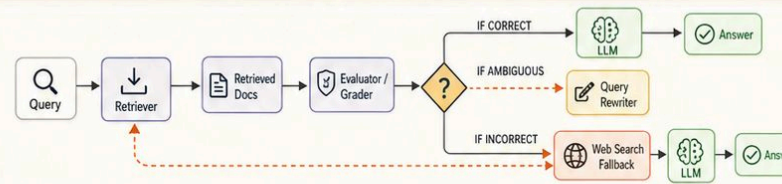
Retrieval becomes a plan, not a step.



04

CORRECTIVE RAG (CRAG)

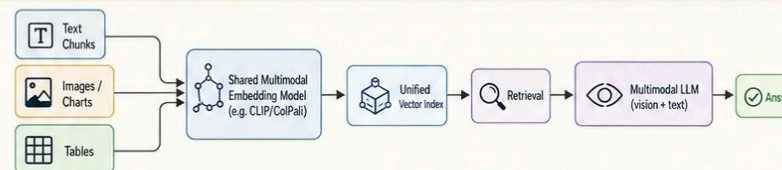
Grade the retrieval before you trust it.



05

MULTIMODAL RAG

One index across text, images, and tables.



The 8 Memory Layers Behind Reliable AI Agents

Skip one, and your agent breaks in production.

01 Working Memory

It forgets the start of a long task. Silent truncation breaks long workflows.



Fix Manage the window, summarize early, stop trusting silent truncation.

02 Episodic Memory

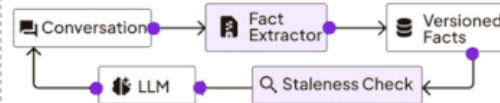
It pulls up the wrong past conversation. Irrelevant or stale retrievals.



Fix Tighten retrieval, filter by recency and relevance, prune noisy chunks.

03 Semantic Memory

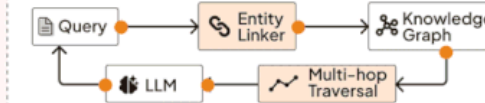
It repeats a fact that's no longer true. Incorrect facts persist unnoticed.



Fix Version stored facts and add a staleness check before they're reused.

04 Entity Memory

It can't connect two things it clearly knows. Brittle links across people and events.



Fix Add a knowledge graph so the agent can reason across people and events.

05 Procedural Memory

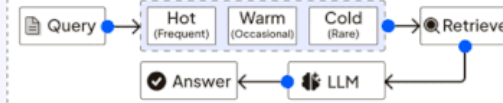
It relearns the same workflow every time. Flawed workflows promoted without validation.



Fix Capture successful runs as reusable steps, but validate before promoting.

06 Hierarchical Memory

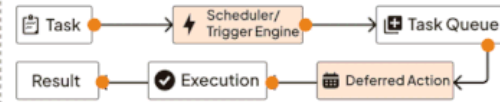
It slows to a crawl at scale. Latency spikes during retrieval.



Fix Tier storage into hot, warm, cold so every lookup doesn't cost full price.

07 Prospective Memory

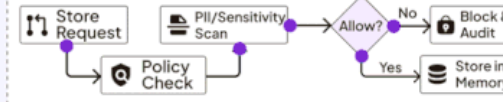
It drops the follow-up it promised. Missed triggers without reliable scheduling.



Fix Schedule commitments with fault-tolerant triggers, not hope.

08 Governance Memory

It surfaces something it should never have stored. Stale policy becomes a compliance risk.



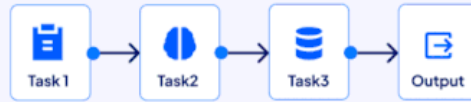
Fix Set policy on what the agent may store and surface before it goes live.

Multi-Agent Design Patterns

A Cost-Value Breakdown

1 Sequential

Cost: \$



Best For

Deterministic workflows with clear dependencies.

Hidden Cost

Slower execution when tasks could run in parallel.

Use When

Simplicity, reliability and low overhead are the priorities.

2 Parallel

Cost: \$ \$



Best For

Reducing latency for independent tasks at scale.

Hidden Cost

Higher token usage, compute duplication and infrastructure cost.

Avoid When

Tasks depend on each other or share context heavily.

3 Hierarchical

Cost: \$ \$ \$



Best Long-Term ROI

Reusable sub-agents create leverage across workflows.

Hidden Cost

Complex orchestration layer and higher development effort.

Wins At

Enterprise scale with repeatable use cases.

4 Generator-Critic

Cost: \$ \$ \$



Best For

High-quality content, code, plans and complex outputs.

Hidden Cost

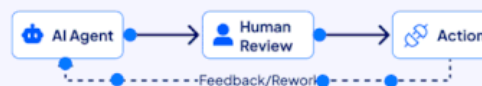
Multiple inference loops increase latency and cost.

Worth It When

Accuracy and quality matter more than speed.

5 Human-in-the-Loop

Cost: \$ \$ \$



Best For

Decisions with real consequences or audit requirements.

Hidden Cost

Intentional latency and manual bandwidth.

Non-Negotiable For

Compliance, billing, legal, safety and high-risk decisions.

6 Composite

Cost: \$ \$ \$ \$



Best For

Handles complex, real-world end-to-end automation.

Hidden Cost

Highest infrastructure and orchestration overhead.

Biggest Mistake

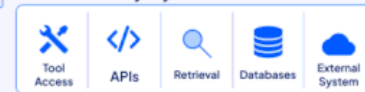
Using it too early or for problems that don't need it.

The 2026 Production Stack



MCP

Tool Connectivity Layer



Works with



A2A

Agent Coordination Layer



Works with



@rakeshgoel01

20 LLM FINE-TUNING TECHNIQUES

Greg Coquillo
Product Leader

Powerful Methods To Adapt, Align & Optimize
Large Language Models For Your Needs



LoRA

- Learns low-rank updates while keeping base weights frozen.
- ~90% fewer trainable parameters.



QLoRA

- Applies LoRA on 4-bit quantized models.
- Enables large models to fit on a single GPU.



Prefix Tuning

- Adds trainable prefix vectors to each layer.
- Core model weights remain unchanged.



Adapter Tuning

- Trains small adapter modules between layers.
- Parameter-efficient transfer learning.



Instruction Tuning

- Fine-tunes on instruction-response pairs.
- Improves ability to follow user commands.



P-Tuning

- Learns continuous prompt embeddings.
- Stronger results on NLP tasks.



BitFit

- Updates only bias parameters.
- Extremely lightweight and efficient.



Soft Prompts

- Uses trainable embeddings as virtual tokens.
- Guides the model without changing weights.



RLHF

- Optimizes using human feedback.
- Combines supervised learning with RL.



RLAI

- Uses AI feedback instead of human labels.
- Cuts labeling cost while maintaining quality.



DPO

- Directly optimizes preference pairs.
- Simpler and more stable than PPO.



GRPO

- Group-based reward optimization.
- Removes value model; reduces memory usage.



RLVR

- Verifies answers with a rule-based checker or compiler.
- Great for code & math tasks.



Multi-Task Tuning

- Trains on multiple datasets together.
- Boosts generalization across different tasks.



Federated Tuning

- Updates are trained locally and shared.
- Keeps data on-device; protects privacy.



Continued Pretraining

- Further trains on domain-specific unlabeled data.
- Improves fluency & domain adaptation.



Domain-Adaptive Pretraining

- Continued pretraining on domain-relevant data.
- Stronger knowledge in specific domains.



Data Selection

- Selects high-quality, relevant data for training.
- Improves efficiency and reduces noise.



Curriculum Learning

- Trains from easy to hard examples.
- Helps model learn more stable & effectively.



Parameter-Efficient Full Fine-Tuning

- Updates all model parameters during training.
- Provides maximum flexibility but requires more compute.



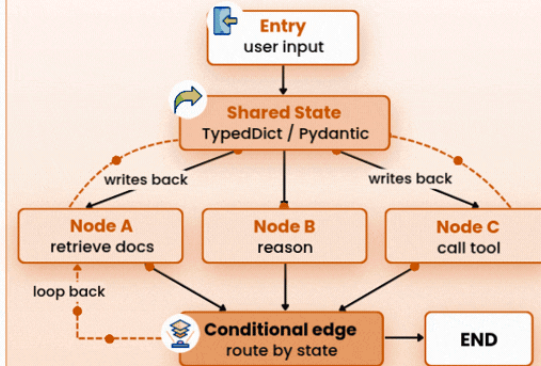
Pro Tip: Start simple (LoRA / QLoRA / Instruction Tuning) and choose techniques based on your task, data, and compute budget.

AI AGENT FRAMEWORKS: THE COORDINATION PLAYBOOK

LangGraph — State Machine



Shared state object flows through a directed graph. Nodes read and update state; edges route execution based on state values. Checkpointer persists every step

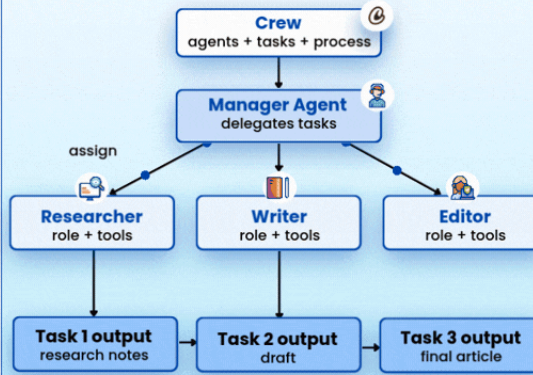


Checkpointers persist state — pause, resume, replay any step

CrewAI — Role-Based Org Chart



Agents have roles, goals, and backstories. A manager delegates tasks to specialists; outputs flow sequentially through a task pipeline.

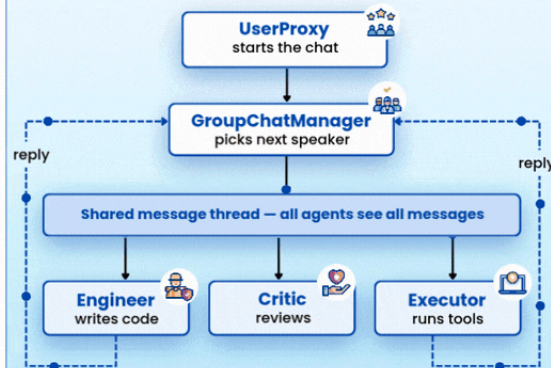


Role-goal-backstory composes the system prompt at runtime

AutoGen — Group Conversation



Agents exchange messages in a shared thread. A GroupChatManager picks who speaks next — round-robin, LLM-auto, or custom function.

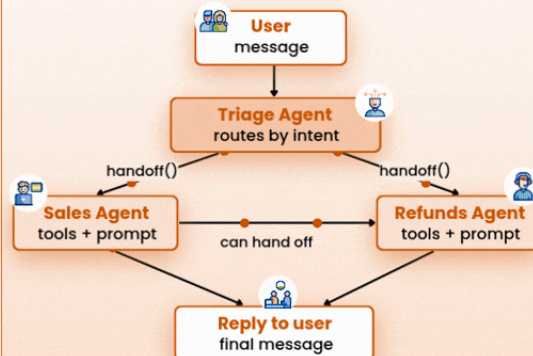


One speaks per turn — manager loops until termination condition

OpenAI Swarm — Handoff Chain

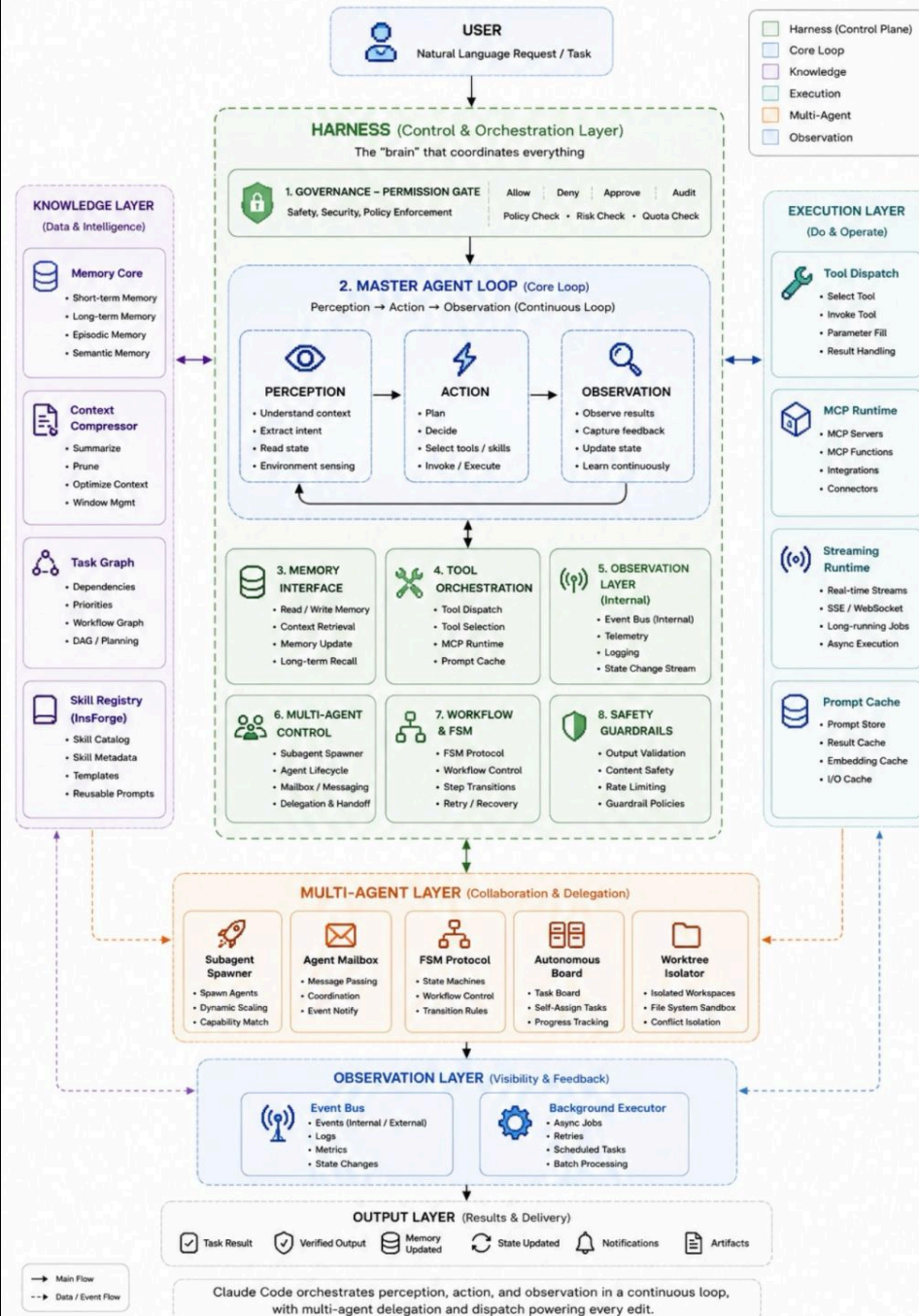


Stateless. Each agent is instructions plus functions; a function that returns an Agent transfers control. No graph, no shared state, just handoffs.

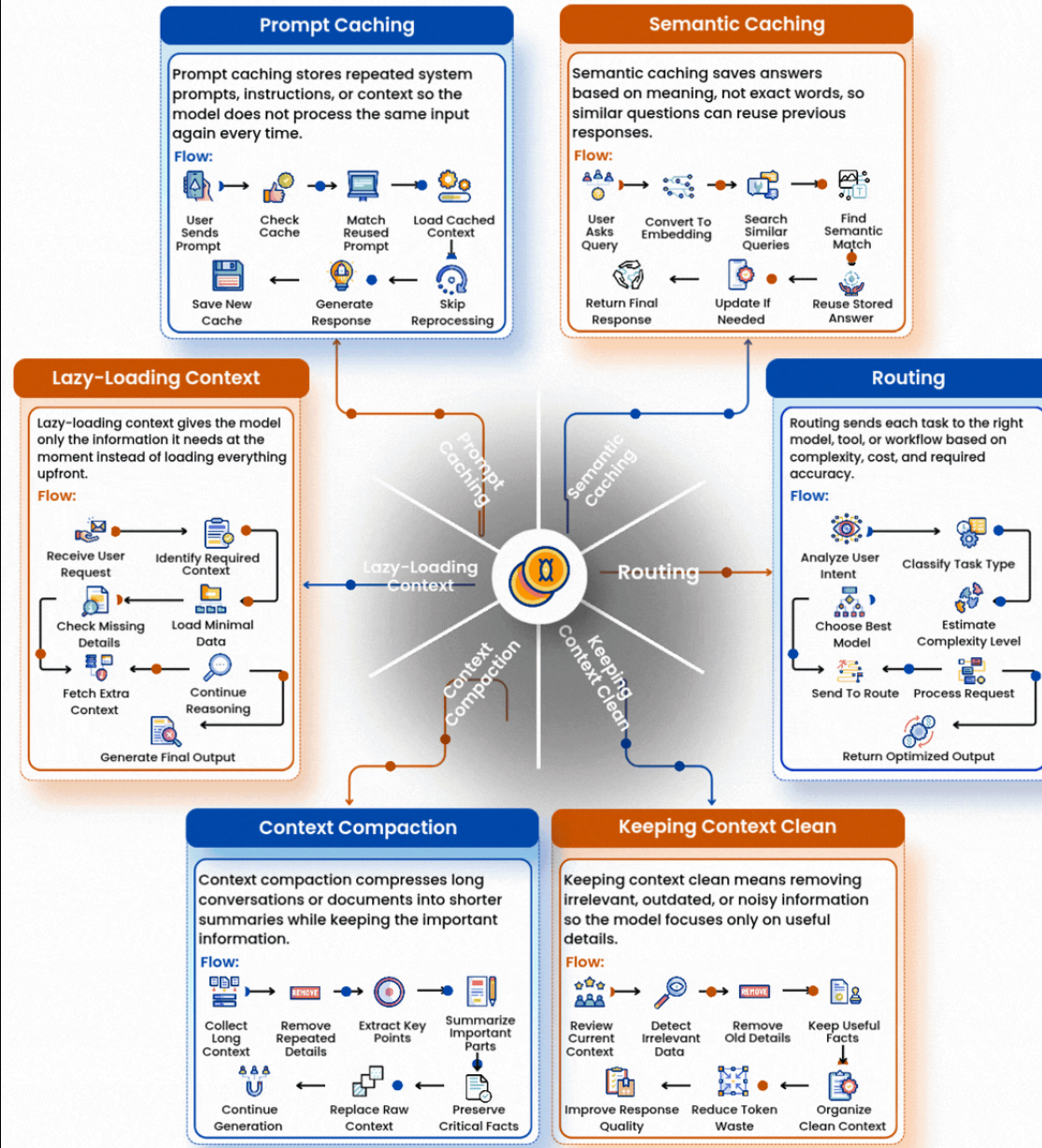


Stateless — context_variables thread through tool calls

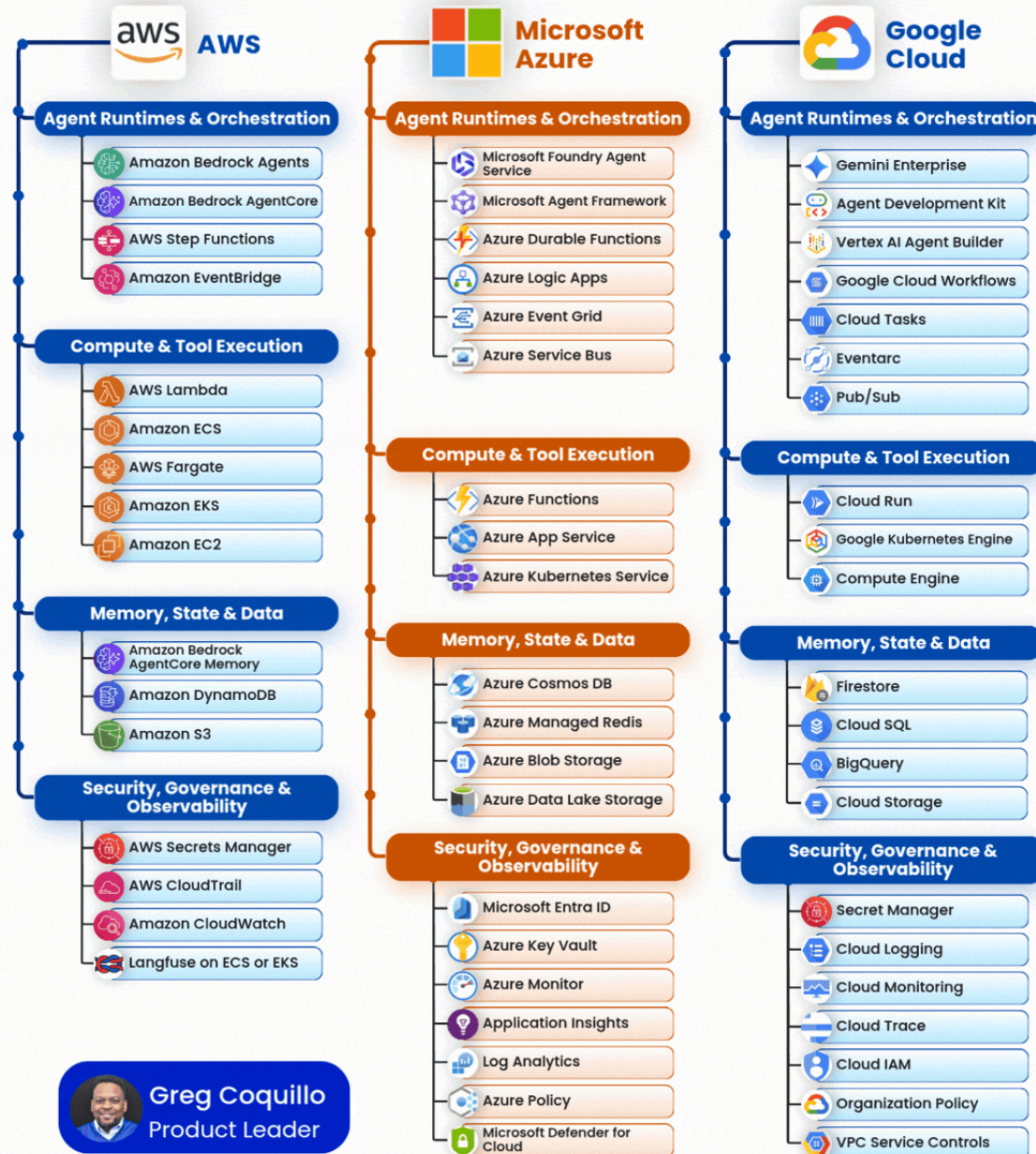
CLAUDE CODE ARCHITECTURE



AGENTIC AI: HOW TO SAVE ON TOKENS

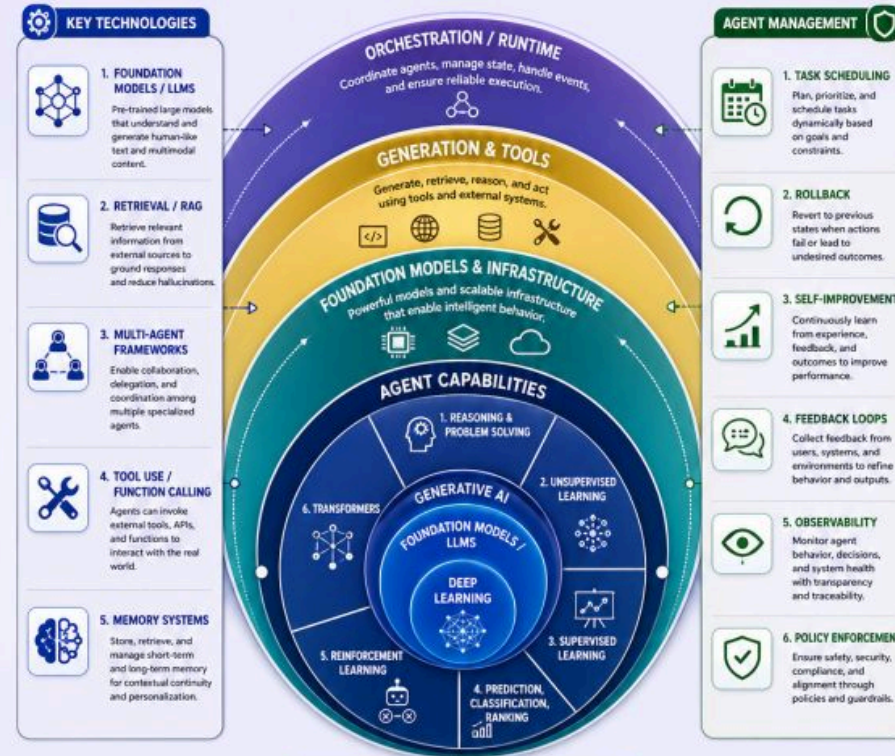


AGENTIC AI STACK ACROSS AWS, AZURE, AND GCP

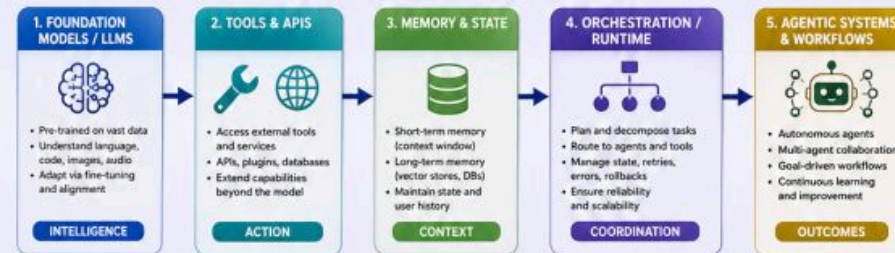


AGENTIC AI: A COMPLETE FRAMEWORK

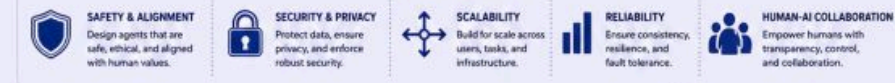
A layered approach to building intelligent, autonomous, and adaptable AI agents that perceive, reason, act, and continuously improve.



THE AGENTIC AI STACK - AN END-TO-END VIEW



GUIDING PRINCIPLES



Naresh Hingorani

THE MODERN AI ECOSYSTEM - TOOLS

